

Final Report for CS120

Changgui Xu and Zihao Xu

xuchg2022@shanghaitech.edu.cn

xuzh2022@shanghaitech.edu.cn

I. Project 1: Acoustic Link

A. Stories

Task1 and Task2 mainly focus on the principles of signal modulation methods and how to use code to emit and receive sound. This part is easy to understand. But Task3 requires us to accurately transmit a large amount of bit signals between two computers. At first, we encapsulated 10,000 bits into a frame and sent it, but due to the sound signal passing through the air medium, a large amount of data was lost. Later, we split it into 100 frames according to SamplePHY.m and successfully collected sufficient data, but the accuracy rate was only 50%. Finally, we tried multiple parameters and used various devices such as microphones and amplifiers, successfully increasing the accuracy rate to 100%.

B. Designs

- Initially, FSK was tested but found unstable due to environmental noise. The team switched to PSK for improved reliability, as it better handles amplitude variations.
- To address low decoding accuracy in the middle of large frames, the data was divided into 100 smaller frames, each containing 100 bits. Each frame included a preamble for synchronization and CRC for error detection. This structure prevented mid-frame signal loss.
- At the receiver, each frame's data portion was strictly decoded as 100 bits to avoid global misalignment caused by bit errors.
- A high-pass filter was applied to eliminate low-frequency noise. However, since high-frequency noise affected preamble detection, additional dB-based filtering was used to distinguish noise from valid signals.
- The quality of microphones and speakers was emphasized, as better equipment significantly improved signal recognition accuracy.
- Guard intervals were added between symbols to reduce interference caused by the limited sampling rate in 15-second transmissions.

C. Challenges and Solutions

- *Modulation Scheme Instability*: The initially adopted FSK modulation was susceptible to environmental noise and highly unstable, necessitating a switch to PSK.
- *Frame Synchronization Errors*: Large frames (10,000 bits) led to very low decoding accuracy in the middle segments, so we segmented 10,000 bits into 100 smaller frames of 100 bits each, each with a preamble and CRC check, to prevent mid-signal loss and decoding misalignment.
- *Inter-Symbol Interference (ISI)*: High transmission rates within a short duration caused severe ISI, so we increased the guard interval to mitigate severe ISI caused by high transmission rates in a short time.
- *Equipment Dependence*: The quality of recording devices and speakers significantly impacted signal recognition accuracy.

D. Suggestions

- *Add necessary prompts*: Although Project1 is not difficult, there is a certain gap between the parts it needs to implement and the theoretical knowledge taught in class, which makes it very hard for us to get started at the beginning.

- *Reduce the impact of equipment:* When the accuracy rate was only 50%, we spent a lot of time debugging parameters and processing sound signals, but in the end, there was no effect. In the end, we merely replaced the equipment and easily increased the accuracy rate to 100%. This indicates that different devices have a significant impact on this project, and this is something we cannot confirm through debugging.

II. Project 2: Manage Multiple Access

A. Stories

Unlike Project1, Project2 requires two computers to transmit data via audio cables, which should have higher transmission accuracy. However, when we were working on Task1, we found that there was still considerable noise interference at the receiving end. Therefore, we used the OBS library to improve the audio quality. In Task2, due to the relatively strict transmission time limit, we were unable to process the audio too much, resulting in multiple retransmissions to ensure accuracy. Ultimately, we were never able to complete 100% accuracy transmission within the scoring time limit.

B. Designs

- For the cable connection, the implementation likely simplified the physical layer. While the document suggests that complex carrier waves were no longer necessary, we used PSK modulation and a frame structure with a preamble for synchronization and CRC for error detection, building on the P1 experience.
- The importance of using high-quality audio equipment was emphasized. A critical safety measure was to unplug power adapters during connection to prevent ground loop issues that could cause DC bias or hardware damage.

C. Challenges and Solutions

- *Low Transmission Speed:* The task could not be fully completed (only Task1 finished). The primary cause was the slow transmission rate, which caused timeouts for Task2 and subsequent node interactions.
- *Weak and Noisy Audio Signal:* The computer's audio output was identified as too weak and noisy through monitoring, leading to low preamble recognition and frequent decoding errors. This hardware limitation could not be overcome.
- *Python Code Limitations:* Timestamping revealed that opening/closing audio streams and reading from the sound card incurred significant time overheads. This was identified as a limitation with no viable solution found.

D. Suggestions

- *Relax the scoring requirements:* Since the language used by our group is Python, there is an inevitable large latency, so we spent a long time optimizing RTT.

III. Project 3: To the Internet

A. Stories

Overall, the completion of Project3 this time was very hasty. Due to the arrival of various mid-term exams, we don't have enough time to complete this project. At first, we were troubled by how to use an audio cable to transmit ICMP Echo. Subsequently, during the implementation process, we found that the RTT was always very large due to the limitations of the python code. We tried various methods, such as reducing the number of sampling points for each symbol, shortening the header length of ICMP Echo, and ignoring the checksum at the receiving end, and successfully kept the RTT within 200ms. In the implementation of Task2, NODE2 has been able to successfully send ICMP Reply to NODE3, but due to device issues, the sending port cannot be specified by itself, resulting in NODE3's failure to receive successfully.

B. Designs

- A significant improvement was changing the preamble from a linear carrier wave to a square wave. This modification drastically increased the accuracy of preamble detection at the receiver, which was crucial for reliable frame synchronization.
- A major challenge was the unreliable decoding of long data packets, where the signal strength weakened towards the end. The implemented solution was to strictly limit the length of transmitted data packets to no more than 10 bytes. This avoided the issue of the last few bytes being incorrectly decoded. This short packet size was a practical compromise to ensure reliability over the audio link, even though it would impact efficiency for higher-layer protocols.
- The implementation on NODE2 involved adding basic IP packet forwarding logic. If a packet's destination IP belonged to a directly connected network (like the Aethernet with NODE1 or the Wi-Fi with NODE3), it was forwarded to the corresponding port.

C. Challenges and Solutions

- *Preamble Detection:* The initially low preamble recognition accuracy was improved by changing the carrier wave from a linear wave to a square wave.
- *Signal Attenuation:* To address the issue where only the first few bytes of a received data packet could be accurately identified (likely due to signal attenuation), we compressed the length of the data packet sent by the transmitter to no more than 10 bytes.
- *Network Interface Card (NIC) Limitation:* Due to the limitation of the NIC on NODE2, which incorrectly forwarded all packets to the same port, we manually implemented a forwarding table to complete Task2.

D. Suggestions

- *Reduce the impact of equipment:* Same as Project1, due to the limitations of NIC, we were ultimately unable to complete Task2. Therefore, we hope to be able to reduce our reliance on the device.
- *Software Tutorial:* This project, several Network software such as Wireshark and Network Tools were used, but we were not familiar with them, which made it very difficult to use.

IV. Project 4: Above IP

A. Stories

Project4 requires the implementation of DNS protocol, UDP protocol and HTTP protocol. With the experience gained from Project3, this project went much more smoothly. The only problem we have is that if the transmitted signal is too long, the last part of the signal may fail to be parsed. So we specifically simplified the signal composition, retaining only the key parts, and output a silent zero at the end to make the sound signal smoother and more stable. At the same time, we also added a monitoring window. Ultimately, our results were quite good, with both the accuracy rate and the reception length being well guaranteed.

B. Designs

- *Task 1: DNS Implementation.* Upgrade the NAT on NODE2 to support TCP and UDP protocols, and configure a local DNS server on NODE2. This allows NODE1 to resolve domain names recursively.
- *Key Implementation Details:* The local DNS server on NODE2 handles iterative queries to external DNS servers (e.g., 10.15.44.11 or 1.1.1.1) on behalf of NODE1. In the Ethernet connection between NODE1 and NODE2, the UDP header was simplified (reduced from 8 bytes to 1 byte) to improve reliability over the audio link. A custom end symbol was defined to mark packet boundaries, avoiding the need for checksums or length fields.
- *Task 2: HTTP Implementation.* Implement a simplified TCP handshake and sliding window mechanism on NODE1 to fetch HTML pages via HTTP. A curl-like command was integrated into the Aethernet program.

- *Key Implementation Details:* The TCP implementation includes a full three-way handshake and four-way disconnection to manage connections. Python libraries such as urllib were used to handle HTTP GET requests and parse responses. To reduce packet length issues, only the domain name is transmitted from NODE1 to NODE2 over the audio link, and NODE2 handles the actual HTTP request using its network connection.

C. Challenges and Solutions

- *Parameter Sensitivity:* Extensive parameter tuning was required due to equipment sensitivity. We conducted extensive debugging to find a set of parameters suitable for the equipment and providing good accuracy.
- *Accuracy vs. Efficiency:* To achieve high accuracy, we used a long listening window (15s) when no time constraint existed, sacrificing real-time performance. This involved recording for a long duration and then searching for preambles and packets in the sample sequence.
- *Data Packet Design:* Long data packets (e.g., HTTP responses) often had the last bytes misdecoded as 0xf. We added hundreds of bytes of silent zeros at the end to improve recognition accuracy, but the issue was considered probabilistic and not resolved.
- *Protocol Simplification:* To improve efficiency and reliability over the unreliable Ethernet connection, we used simplified packet headers (reduced from 8B to 1B) for NODE1-NODE2 communication, and defined a special termination symbol to eliminate the need for checksums and length fields. For HTTP requests, we implemented a complete three-way handshake and four-way handshake to ensure correct sequence numbers. A genuine UDP packet was constructed only for DNS queries during forwarding.

D. Suggestions

- *Clarify the task requirements:* At the beginning, we were troubled by the problem for a long time. For example, since NODE1 and NODE2 are transmitted through audio cables, what should be looked at when capturing packets through Wireshark on NODE2? Later, by asking the teaching assistant, I understood what we needed to do. I hope the next question can be explained more clearly.